

Parallelizing DSP Workloads

FFT and FIR Filtering Across Pthreads, OpenMP, and MPI

Vaidehi Gohil

Anthony Yalong

EECE 5640: High Performance Computing

Northeastern University

April 2026

Contents

1	Introduction	2
2	Background	2
2.1	Fast Fourier Transform	2
2.2	FIR Filtering	3
2.3	Parallelization Middlewares	3
2.4	Amdahl's Law and Parallel Efficiency	4
3	Experimental Setup	5
3.1	Hardware and Cluster Environment	5
3.2	Input Datasets	5
3.3	Measurement Methodology	5
4	Implementation	6
4.1	Serial Baselines	6
4.2	FFT Parallelization	6
4.3	FIR Parallelization	7
5	Results and Analysis	8
5.1	Serial Baseline Characterization	8
5.2	Strong Scaling: Speedup and Efficiency	8
5.3	FFT vs FIR Comparison	11
5.4	VTune Profiling Analysis	12
5.5	Weak Scaling	14
5.6	MPI vs Shared-Memory Comparison	15
5.7	Generated vs Downloaded Input	16
6	Conclusion	16
	Code Availability	17
	Acknowledgments	18
	References	18

1 Introduction

Digital signal processing (DSP) workloads are among the most computationally demanding tasks in modern audio, communications, and scientific computing pipelines. Two foundational DSP algorithms, the Fast Fourier Transform (FFT) and Finite Impulse Response (FIR) filtering, are ubiquitous in these domains and represent structurally distinct parallelism profiles. The FFT, based on the Cooley-Tukey divide-and-conquer algorithm, exhibits recursive data dependencies that constrain parallelization. FIR filtering, which computes each output sample as an independent dot product of filter coefficients with a window of input samples, is inherently parallel. Studying both workloads in tandem allows a direct comparison of how algorithmic structure governs parallel scaling behavior.

This project evaluates both workloads across three parallelization middlewares: POSIX Threads (Pthreads), OpenMP, and OpenMPI. Pthreads provides low-level manual thread management with explicit synchronization; OpenMP offers compiler-directive-based shared-memory parallelism with a higher-level programming model; and MPI enables distributed-memory parallelism via message passing, included here as an extra-credit extension beyond the core shared-memory analysis. Each middleware represents a different point in the tradeoff space between programmer control, runtime overhead, and scalability. All experiments are conducted on the Explorer Cluster at Northeastern University.

Experiments span ten input sizes ranging from 2^{20} to 2^{29} samples, using both synthetically generated audio signals and real-world recordings from the LibriSpeech corpus. For each configuration, we measure strong scaling speedup and parallel efficiency, conduct weak scaling analysis, and use Intel VTune Profiler to identify hotspots and characterize hardware-level parallelism. The central finding is that algorithmic structure dominates scaling behavior: FIR achieves near-ideal scaling across all middlewares and sizes, while FFT’s serial `bit_reverse` permutation step imposes a hard Amdahl limit that shared-memory middlewares cannot overcome, a limit that MPI’s distributed decomposition substantially mitigates.

2 Background

2.1 Fast Fourier Transform

The Fast Fourier Transform is an algorithm for computing the Discrete Fourier Transform (DFT) of a sequence of N samples in $O(N \log N)$ time, compared to the $O(N^2)$ cost of the naive DFT. The Cooley-Tukey algorithm, the most widely used FFT formulation, achieves

this by recursively decomposing the DFT of size N into two DFTs of size $N/2$, exploiting the periodicity and symmetry of complex exponentials. This divide-and-conquer structure repeats for $\log_2 N$ stages, with each stage performing butterfly operations across the entire input array.

Parallelizing the FFT is non-trivial. The butterfly operations within each stage are independent and can be distributed across threads, but successive stages depend on the outputs of the previous stage, requiring synchronization between them. Additionally, the algorithm requires a bit-reversal permutation of the input array prior to computation. This permutation is inherently sequential in standard implementations and cannot be parallelized without significant restructuring. As the number of threads increases, the cost of this serial section becomes the dominant bottleneck, a behavior well-predicted by Amdahl's Law.

2.2 FIR Filtering

A Finite Impulse Response filter applies a linear time-invariant filter to a signal via discrete convolution. Given an input signal $x[n]$ and a filter with M coefficients $h[k]$, each output sample $y[n]$ is computed as:

$$y[n] = \sum_{k=0}^{M-1} h[k] \cdot x[n - k] \quad (1)$$

Each output sample depends only on a fixed window of M input samples and the filter coefficients, with no dependency on other output samples. This makes FIR filtering inherently parallel: the computation of every output sample is fully independent and can be assigned to any thread or process without synchronization. The filter coefficients used in this project are generated using the Hamming window method to construct a low-pass filter with 101 taps.

2.3 Parallelization Middlewares

Three parallelization middlewares are evaluated in this project. POSIX Threads (Pthreads) is a low-level threading API that gives the programmer explicit control over thread creation, synchronization, and destruction. Threads are created with `pthread_create` and joined with `pthread_join`, with work distributed manually by partitioning the input array. This approach minimizes runtime overhead but places the full burden of correctness and load balancing on the programmer.

OpenMP is a compiler-directive-based shared-memory parallelism model. Parallel regions

and loop distributions are specified via pragmas, with the runtime managing thread creation, scheduling, and synchronization automatically. OpenMP reduces programming effort significantly and allows the compiler to apply additional optimizations, but introduces some runtime overhead from its scheduling and barrier machinery.

OpenMPI implements the Message Passing Interface (MPI) standard for distributed-memory parallelism. Each process operates on an independent memory space, communicating explicitly via send and receive operations. In this project, MPI is used to distribute disjoint chunks of the input across processes, with each process computing its local FFT or FIR result independently. MPI incurs communication overhead at process boundaries but avoids the shared-memory synchronization costs of Pthreads and OpenMP, and can in principle scale beyond a single node.

2.4 Amdahl's Law and Parallel Efficiency

Amdahl's Law provides a theoretical upper bound on the speedup achievable by parallelizing a program with a fixed serial fraction. If a fraction f of a program's execution time is inherently serial, the maximum speedup achievable with p processors is:

$$S(p) = \frac{1}{f + \frac{1-f}{p}} \quad (2)$$

As p approaches infinity, the speedup converges to $1/f$ regardless of how many processors are added. This places a hard ceiling on scalability for any algorithm with a non-trivial serial component.

Parallel efficiency $E(p) = S(p)/p$ measures how effectively the available processors are being utilized, with $E(p) = 1.0$ representing ideal linear scaling. In practice, efficiency degrades with increasing thread count due to synchronization overhead, load imbalance, and serial bottlenecks. Throughout this report, speedup is computed as $S(p) = T(1)/T(p)$, where $T(1)$ is the serial baseline execution time for Pthreads and OpenMP, and the single-process MPI execution time for MPI, reflecting the different computational models of the shared-memory and distributed implementations.

3 Experimental Setup

3.1 Hardware and Cluster Environment

All experiments were conducted on the Explorer Cluster at Northeastern University using the course reservation for EECE 5640. Jobs were submitted via SBATCH scripts to a single multi-core compute node. The shared-memory experiments (Pthreads and OpenMP) ran on node `d0125`, and the MPI experiments ran on node `d0135`, with the serial baseline also collected on `d0125`. Thread and process counts were swept across 1, 2, 4, 8, 16, and 32. All implementations were compiled with GCC using optimization flag `-O2` and linked against the math library. The OpenMPI implementation was compiled with `mpicc` under the `openmpi` module available on Explorer. Intel VTune Profiler was loaded as a module and used to collect CPU utilization, thread-level parallelism, and hotspot data for the Pthreads and OpenMP implementations.

3.2 Input Datasets

Two categories of input were used across ten sizes ranging from 2^{20} to 2^{29} samples. The first category consists of synthetically generated signals, constructed as a superposition of sine waves at audio frequencies (440 Hz, 880 Hz, 1200 Hz, 2500 Hz, and 5000 Hz) with a small amount of Gaussian noise added to simulate real-world conditions. All generated signals are sampled at 16 kHz and stored as binary `float32` files. The second category consists of real-world audio recordings sourced from the LibriSpeech ASR corpus, specifically the dev-clean subset. These recordings were resampled to 16 kHz, converted to mono, and concatenated or tiled as necessary to reach the required sample counts. Both categories were included to verify that input content does not systematically affect runtime, a hypothesis confirmed by the results and discussed in Section 5.7.

For FIR filtering, a low-pass filter with 101 taps was applied to each input. Filter coefficients were generated using the Hamming window method, which produces a well-conditioned filter with controlled spectral leakage.

3.3 Measurement Methodology

Wall-clock execution time was measured using `clock_gettime` with the `CLOCK_MONOTONIC` clock, wrapping only the core computation and excluding file I/O and setup. Each configuration was run five times and results are reported as the mean and standard deviation across those five runs. Speedup is computed as $S(p) = T(1)/T(p)$, where $T(1)$ is the mean serial

baseline time for Pthreads and OpenMP, and the mean single-process MPI time for MPI. Parallel efficiency is computed as $E(p) = S(p)/p$, with 1.0 representing ideal linear scaling.

Weak scaling efficiency is computed as $E_{\text{weak}}(p) = T(1, n_0) / T(p, n_0 \cdot p)$, where $n_0 = 2^{23}$ is the base problem size assigned to a single thread or process. This formulation keeps the per-process workload constant as the process count increases, with ideal weak scaling corresponding to a flat efficiency of 1.0. The weak scaling diagonal is extracted directly from the strong scaling result files without additional benchmark runs, as all required (p, n) pairs were already collected.

VTune profiling was conducted separately for selected configurations at each thread count from 1 to 32, using input size $n = 2^{27}$ as the representative large-input case. The metrics collected include elapsed time, CPU time, physical core utilization, and the top two hotspot functions by CPU time percentage.

4 Implementation

4.1 Serial Baselines

Serial baseline implementations were written in C for both FFT and FIR. The FFT baseline implements the iterative Cooley-Tukey algorithm in-place on a complex float array. The input is first reordered via bit-reversal permutation, then processed through $\log_2 N$ butterfly stages. Each butterfly stage iterates over the array computing twiddle factor multiplications and additions across pairs of elements separated by the current stride. The FIR baseline implements direct-form convolution, iterating over each output sample and computing the dot product of the 101 filter coefficients with the corresponding window of input samples. Both baselines read input from binary `float32` files and write output to binary `float32` files, with timing measured around the core computation only.

Correctness of all parallel implementations was verified by comparing their output against the serial baseline using a maximum absolute error threshold. This check was run as part of the test suite before any timing data was collected.

4.2 FFT Parallelization

The core parallelizable region of the FFT is the butterfly computation across each stage. For each of the $\log_2 N$ stages, the butterfly operations are independent within the stage and can be distributed across threads or processes. The bit-reversal permutation step runs serially

before the parallel butterfly computation begins in all shared-memory implementations.

In the Pthreads implementation, a fixed thread pool is created once at startup and reused across all stages. Each thread is assigned a contiguous chunk of butterfly pairs for each stage and waits at a `pthread` barrier between stages to ensure correctness before proceeding to the next. The barrier synchronization at each of the $\log_2 N$ stage boundaries is the primary source of overhead in the Pthreads FFT.

In the OpenMP implementation, each butterfly stage is parallelized with a single `parallel for` pragma with static scheduling. The implicit barrier at the end of each parallel region serves the same synchronization role as the explicit pthread barrier, but is managed by the OpenMP runtime. The programmer overhead is significantly lower than the Pthreads version, with the core parallel loop requiring only a pragma annotation.

The MPI implementation takes a fundamentally different approach. Rather than parallelizing across butterfly stages of a single full-length FFT, each process receives a contiguous chunk of N/p input samples and computes a complete FFT on its local chunk independently. This avoids inter-process communication during the butterfly computation entirely, at the cost of computing a segmented rather than a global FFT. This distinction is important for interpreting the MPI speedup results: the MPI implementation solves a partitioned version of the problem rather than the identical computation as the shared-memory implementations, which contributes to its substantially higher observed speedup, particularly at small input sizes where cache effects amplify the advantage.

4.3 FIR Parallelization

FIR parallelization is straightforward across all three middlewares due to the inherently parallel nature of the computation. The output array is partitioned into contiguous chunks, with each thread or process responsible for computing its assigned output samples independently. No synchronization is required between chunks during computation.

In the Pthreads implementation, threads are created once and each is assigned a start and end index into the output array. Each thread iterates over its assigned range and computes the convolution sum for each output sample using the shared read-only filter coefficient array and the shared read-only input array. Threads are joined after all chunks complete.

In the OpenMP implementation, the outer loop over output samples is parallelized with a `parallel for` pragma and static scheduling. The filter coefficient array and input array are shared and read-only, requiring no synchronization.

In the MPI implementation, each process receives its chunk of input samples via `MPI_Scatter`, computes the local FIR output independently, and the results are gathered back via `MPI_Gather`. A small overlap region of $M - 1$ samples at each chunk boundary is included to handle the filter boundary condition correctly, where M is the number of filter taps.

5 Results and Analysis

5.1 Serial Baseline Characterization

Table 1 reports the mean execution time and standard deviation of the serial baseline for both workloads across all input sizes. Both FFT and FIR exhibit consistent growth with input size, and the low standard deviations across five runs confirm stable and reproducible timing on the Explorer nodes. The generated and downloaded inputs produce nearly identical runtimes at all sizes, establishing that input content has no effect on baseline performance.

Table 1: Serial baseline mean execution time (ms) for FFT and FIR across input sizes. Generated input, 5 runs per configuration.

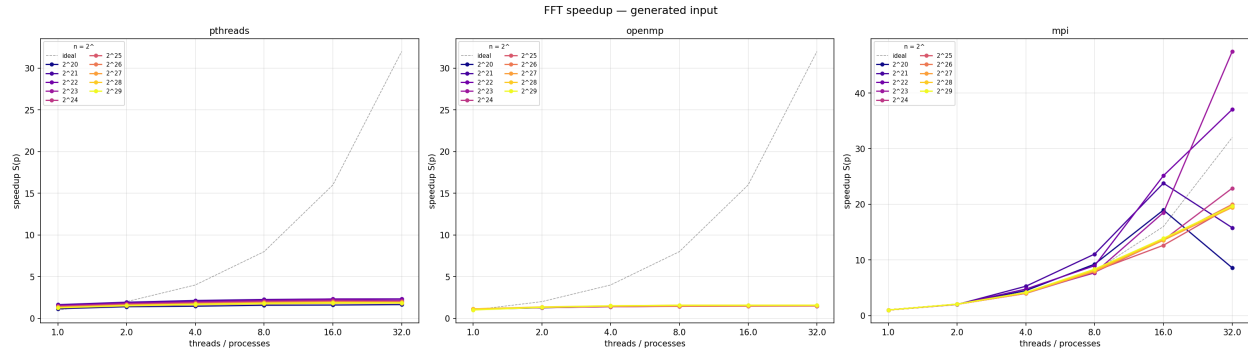
n	FFT		FIR	
	Mean (ms)	Std (ms)	Mean (ms)	Std (ms)
2^{20}	109.78	0.98	97.96	0.47
2^{21}	288.99	2.79	198.13	0.84
2^{22}	619.26	18.27	394.42	2.32
2^{23}	1283.56	13.74	776.71	14.03
2^{24}	2631.14	15.52	1576.67	18.26
2^{25}	5427.75	58.67	3210.13	17.13
2^{26}	11012.81	34.52	6289.24	54.97
2^{27}	22454.85	61.02	12541.88	87.21
2^{28}	46639.49	990.56	25136.62	53.64
2^{29}	96333.87	1666.59	50090.65	98.65

5.2 Strong Scaling: Speedup and Efficiency

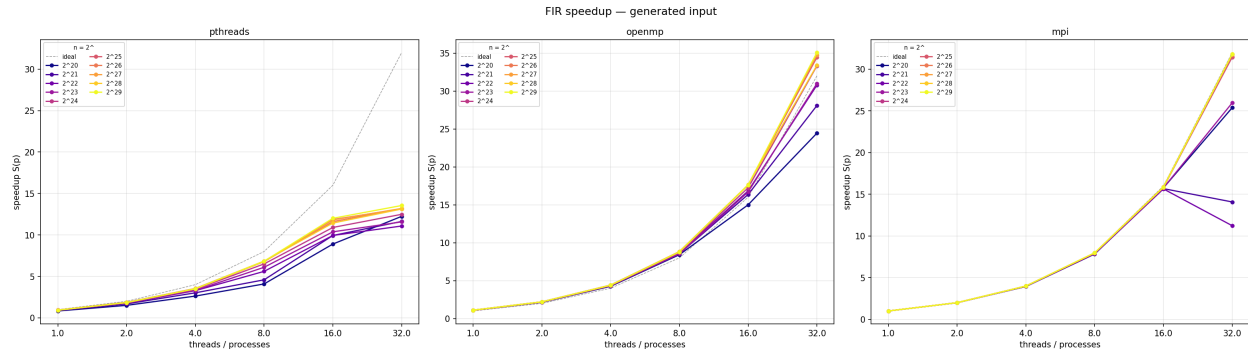
Table 2 reports speedup and parallel efficiency at $n = 2^{27}$ for all middlewares and both workloads. Figures 1 and 2 show speedup and efficiency across all input sizes and thread counts.

Table 2: Speedup $S(p)$ and parallel efficiency $E(p)$ at $n = 2^{27}$, generated input. MPI speedup is relative to MPI at $p = 1$.

FFT					FIR				
Middleware	p	Mean (ms)	$S(p)$	$E(p)$	Middleware	p	Mean (ms)	$S(p)$	$E(p)$
Pthreads	1	16870.68	1.33	1.33	Pthreads	1	13553.35	0.93	0.93
	2	14225.77	1.58	0.79		2	6802.84	1.84	0.92
	4	12948.42	1.73	0.43		4	3581.52	3.50	0.88
	8	12093.54	1.86	0.23		8	1841.55	6.81	0.85
	16	11619.04	1.93	0.12		16	1062.39	11.81	0.74
	32	11667.14	1.92	0.06		32	950.46	13.20	0.41
OpenMP	1	19844.38	1.13	1.13	OpenMP	1	11399.07	1.10	1.10
	2	17019.73	1.32	0.66		2	5701.72	2.20	1.10
	4	15424.23	1.46	0.36		4	2842.34	4.41	1.10
	8	14755.98	1.52	0.19		8	1427.56	8.79	1.10
	16	14607.10	1.54	0.10		16	713.32	17.58	1.10
	32	14560.39	1.54	0.05		32	360.64	34.78	1.09
MPI	1	18596.20	1.00	1.00	MPI	1	10295.46	1.00	1.00
	2	9189.64	2.02	1.01		2	5152.31	2.00	1.00
	4	4611.90	4.03	1.01		4	2580.18	3.99	1.00
	8	2342.17	7.94	0.99		8	1292.46	7.97	1.00
	16	1373.57	13.54	0.85		16	650.44	15.83	0.99
	32	956.82	19.44	0.61		32	323.96	31.78	0.99

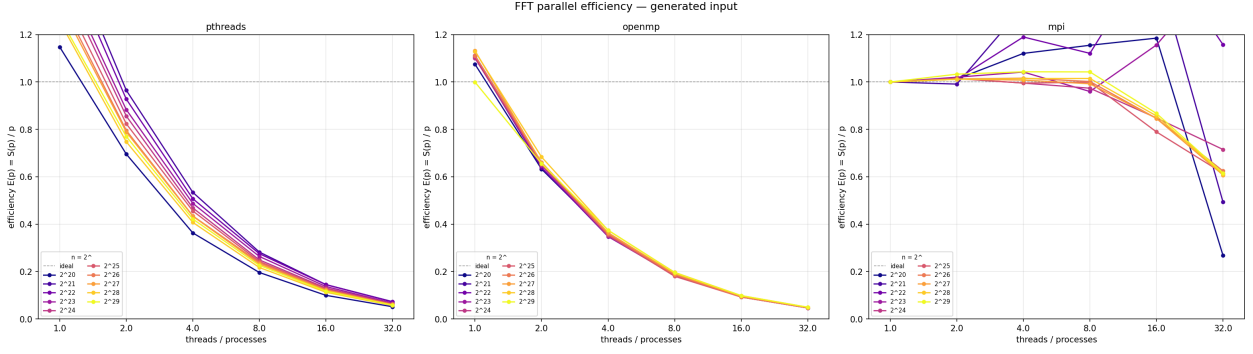


(a) FFT speedup, generated input

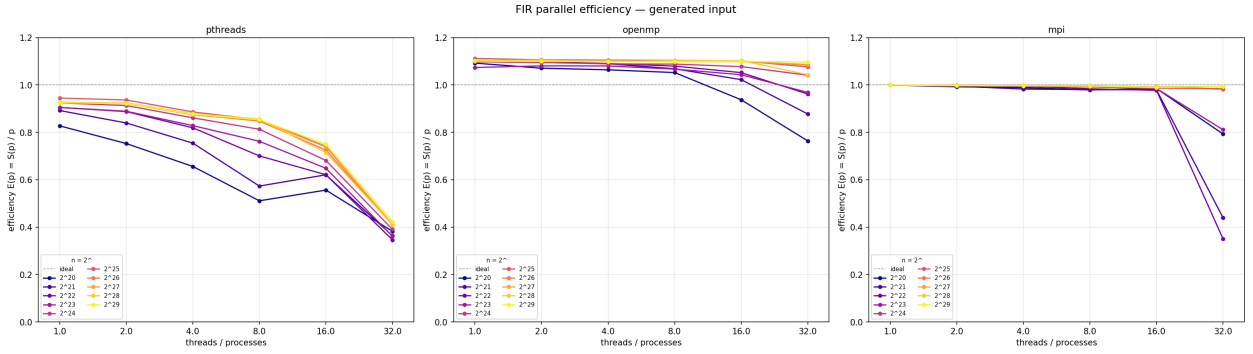


(b) FIR speedup, generated input

Figure 1: Speedup $S(p)$ vs thread/process count for each middleware. Each line represents a different input size from 2^{20} to 2^{29} .



(a) FFT parallel efficiency, generated input



(b) FIR parallel efficiency, generated input

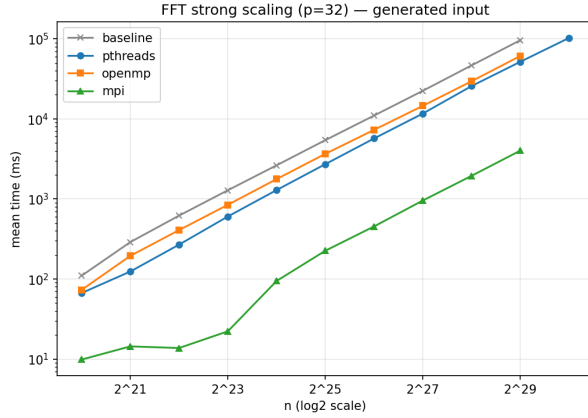
Figure 2: Parallel efficiency $E(p) = S(p)/p$ vs thread/process count for each middleware. Each line represents a different input size from 2^{20} to 2^{29} . Dashed line at $E = 1.0$ indicates ideal scaling.

5.3 FFT vs FIR Comparison

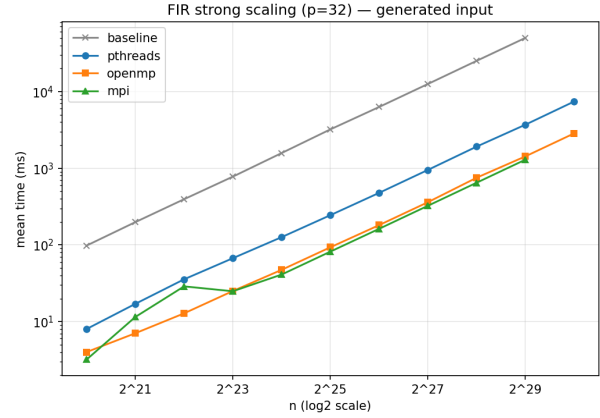
The contrast between FFT and FIR scaling behavior is the central finding of this project. For the shared-memory middlewares, FFT efficiency collapses rapidly: Pthreads and OpenMP both reach below $E = 0.10$ at 32 threads for all input sizes, with speedup plateauing near 1.5 to 2.0 regardless of n . This plateau is a direct consequence of the serial `bit_reverse` step, which accounts for 30 to 60 percent of total CPU time at $n = 2^{27}$ across all thread counts as shown in the VTune hotspot analysis. Amdahl's Law predicts that a serial fraction of even 30 percent limits maximum speedup to approximately 3.3x, consistent with the observed results.

FIR, by contrast, exhibits near-ideal efficiency for OpenMP and MPI at all sizes and thread counts. OpenMP FIR efficiency remains above 1.0 at $n = 2^{27}$ across all thread counts, a result of favorable cache behavior in the parallel convolution loop. MPI FIR tracks near 1.0 throughout. The strong scaling plots in Figure 3 show that at $p = 32$, OpenMP and MPI reduce FIR execution time by roughly 32x relative to the serial baseline, while Pthreads

achieves only 13x due to thread management overhead at high thread counts.



(a) FFT strong scaling, generated input



(b) FIR strong scaling, generated input

Figure 3: Mean execution time vs input size at $p = 32$ for all middlewares and the serial baseline.

5.4 VTune Profiling Analysis

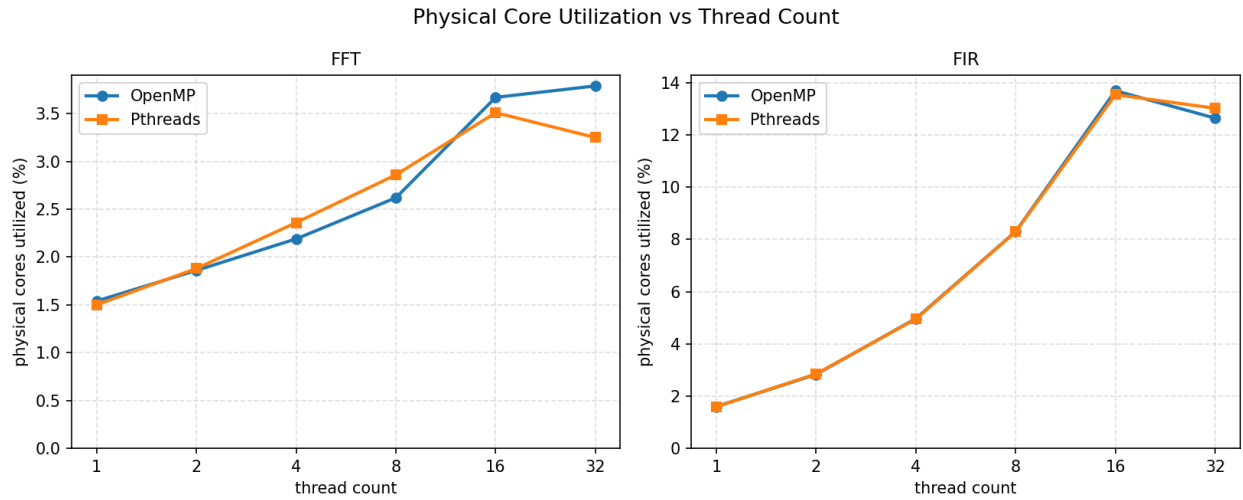


Figure 4: Physical core utilization (%) vs thread count for FFT and FIR, OpenMP and Pthreads. Averaged across input sizes.

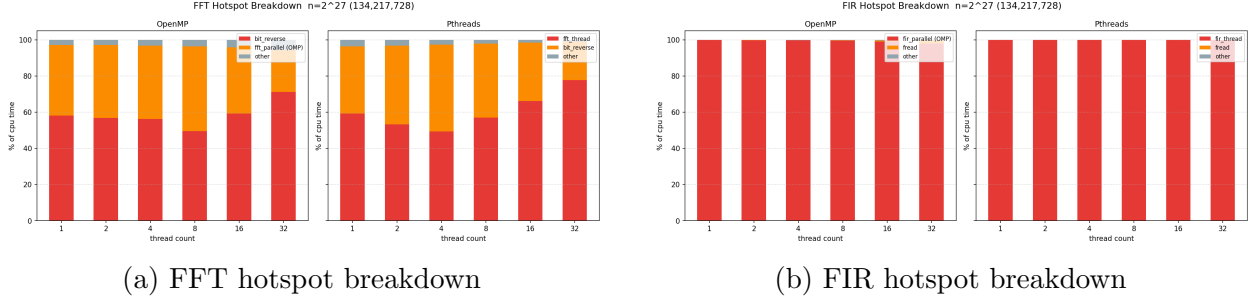


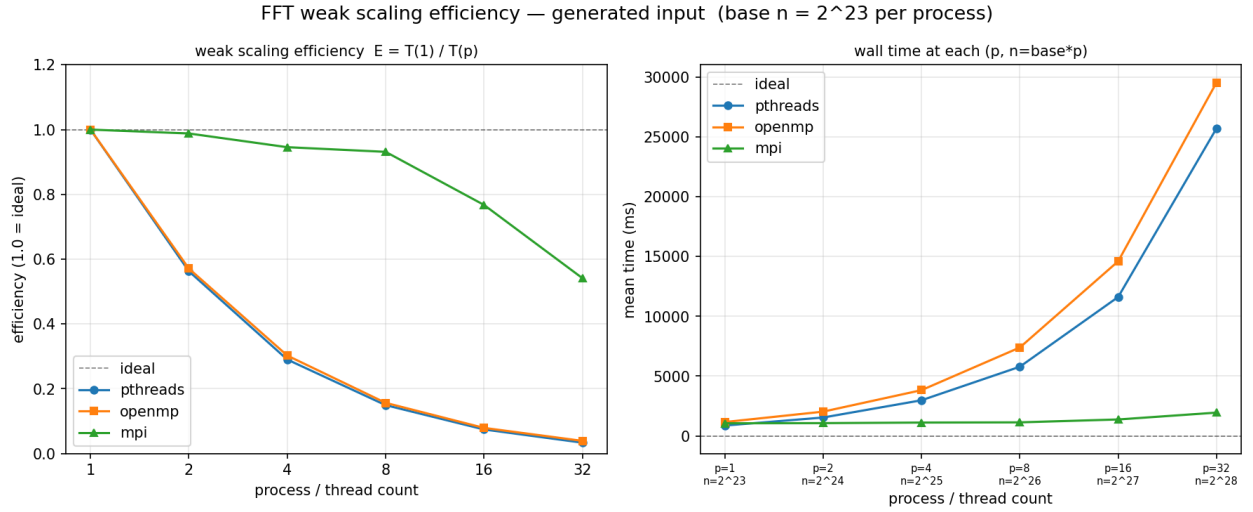
Figure 5: VTune hotspot breakdown at $n = 2^{27}$ across thread counts for OpenMP and Pthreads. Stacked bars show percentage of CPU time attributed to the top two hotspot functions and all others.

The VTune profiling results provide hardware-level evidence for the scaling behavior observed in the timing experiments. Figure 4 shows physical core utilization as a function of thread count for both workloads. FFT reaches a maximum of approximately 3.8 cores utilized at 32 threads, while FIR reaches 13.5 cores at 16 threads before plateauing. The disparity reflects the fraction of time each workload keeps threads active: FFT threads sit idle during the serial `bit_reverse` pass and at stage barriers, while FIR threads remain continuously active across the entire parallel convolution loop.

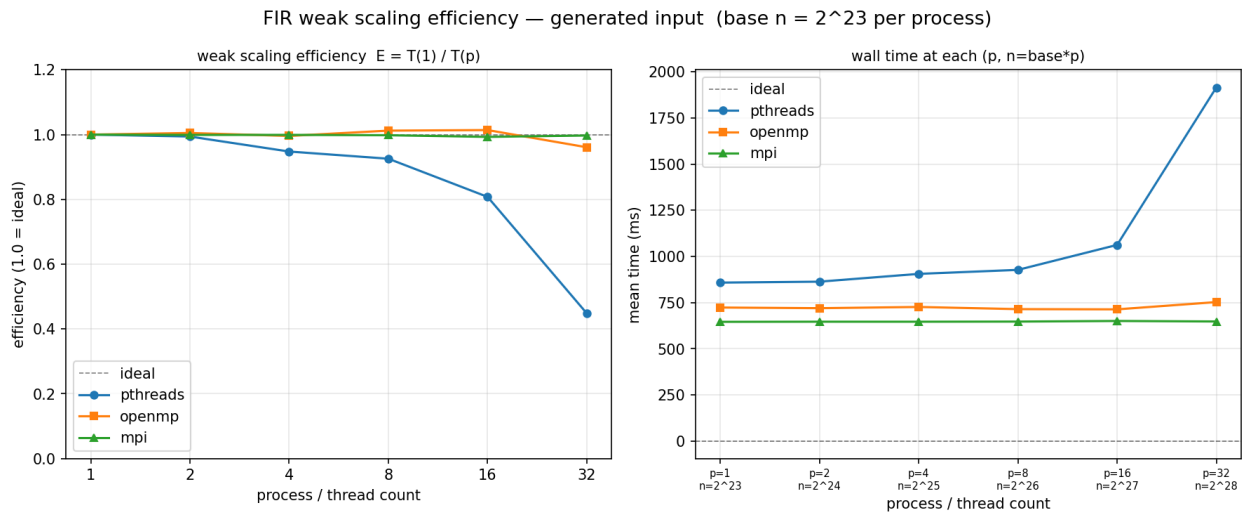
Figure 5 shows the CPU time breakdown by hotspot function at $n = 2^{27}$. At low thread counts, `bit_reverse` accounts for 30 to 60 percent of total CPU time for both OpenMP and Pthreads. As thread count increases, its absolute execution time remains fixed while the parallel butterfly computation completes faster, causing `bit_reverse`'s relative share to shift. At 32 threads, the Pthreads hotspot plot shows `fft_thread` accounting for nearly 80 percent of CPU time with `bit_reverse` reduced to roughly 20 percent. This is precisely the Amdahl dynamic: the serial fraction does not shrink, it simply becomes more exposed as the parallel portion accelerates.

For FIR, `fir_parallel` and `fir_thread` account for essentially 100 percent of CPU time at all thread counts and for both middlewares, with no identifiable serial bottleneck. This confirms that the FIR implementation is spending virtually all of its time in the parallelizable convolution loop and explains the near-ideal scaling efficiency observed in the timing results.

5.5 Weak Scaling



(a) FFT weak scaling, generated input



(b) FIR weak scaling, generated input

Figure 6: Weak scaling efficiency $E_{\text{weak}}(p) = T(1, n_0) / T(p, n_0 \cdot p)$ and raw wall time at each (p, n) pair. Base size $n_0 = 2^{23}$ per process.

Weak scaling analysis holds the per-process problem size constant at $n_0 = 2^{23}$ samples while increasing the total problem size proportionally with process count. Ideal weak scaling corresponds to constant wall time and efficiency of 1.0 across all process counts.

For FFT, Pthreads and OpenMP both collapse immediately: efficiency drops to approximately 0.57 at $p = 2$ and below 0.10 at $p = 32$. The wall time panel shows that their execution time grows nearly as fast as a serial run over the same total n , confirming that adding

threads provides almost no benefit when the problem grows proportionally. This is a direct consequence of `bit_reverse` scaling with total n rather than per-process n : even though each thread's butterfly workload is held constant, the serial permutation step grows with the full input, violating the constant-work-per-process assumption that underlies ideal weak scaling. MPI holds near 0.95 through $p = 4$, degrades to approximately 0.93 at $p = 8$, and reaches 0.55 at $p = 32$. Because each MPI process operates on an independent chunk, `bit_reverse` runs locally on N/p samples rather than on the full N , giving MPI a structural advantage over the shared-memory middlewares in the weak scaling regime.

For FIR, OpenMP and MPI maintain efficiency at essentially 1.0 across all process counts. The wall time panel shows their execution time as nearly flat lines, representing textbook ideal weak scaling. This result confirms that FIR filtering scales perfectly with available parallelism when per-process workload is held constant. Pthreads degrades visibly beginning at $p = 8$ and reaches approximately 0.46 at $p = 32$, with the wall time panel showing a sharp upward turn at high thread counts. Because the computation itself is perfectly parallel, this degradation isolates Pthreads barrier and join overhead as the sole source of inefficiency, a limitation that the OpenMP runtime avoids through more efficient thread management.

5.6 MPI vs Shared-Memory Comparison

The MPI results demonstrate a qualitatively different scaling regime from the shared-memory middlewares, particularly for FFT. At $n = 2^{27}$ and $p = 32$, MPI FFT achieves a speedup of 19.4x compared to 1.92x for Pthreads and 1.54x for OpenMP. This gap widens at smaller input sizes, where MPI reaches superlinear speedup of up to 40x at $n = 2^{22}$ to 2^{23} . This superlinear behavior is attributable to the segmented decomposition: each MPI process operates on a chunk of N/p samples that fits more readily in cache than the full input, yielding cache efficiency gains that compound with the parallelism benefit.

It is important to note that MPI solves a partitioned version of the FFT problem rather than a global FFT. The shared-memory implementations compute a single FFT over the entire input, including the inter-stage butterfly dependencies that require barrier synchronization. MPI avoids this by treating each process's chunk as an independent sub-problem. The performance advantage is therefore not purely a function of parallelism but also reflects a difference in the computation being performed.

For FIR, where the computation is inherently parallel and the problem decomposes cleanly, all three middlewares achieve comparable performance at large n . OpenMP and MPI are essentially tied at $p = 32$ for large inputs, demonstrating that MPI's message-passing overhead

is negligible relative to the FIR computation cost at these sizes. Pthreads remains consistently slower, achieving only 13.2x speedup at $p = 32$ compared to 34.8x for OpenMP and 31.8x for MPI.

5.7 Generated vs Downloaded Input

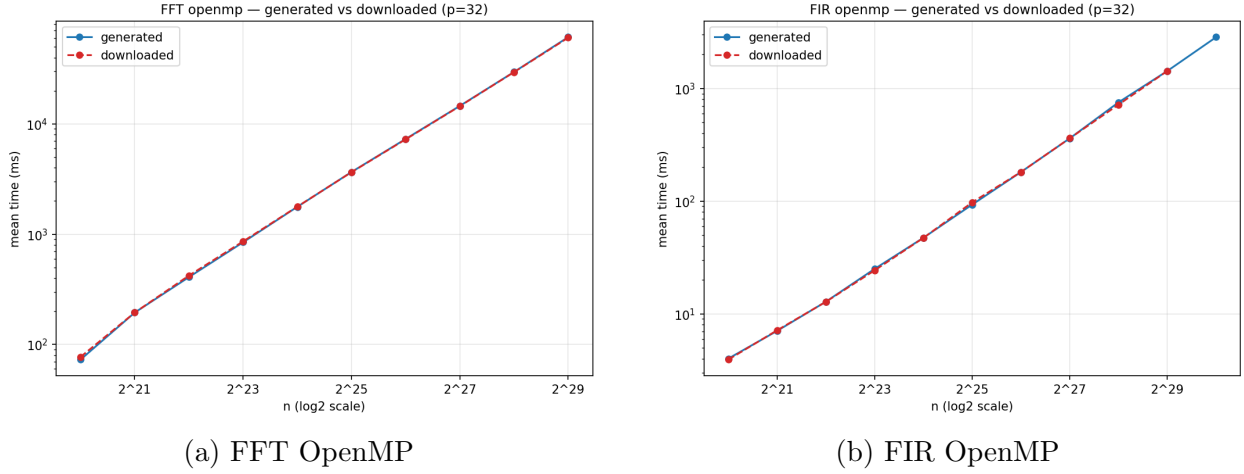


Figure 7: Mean execution time for generated vs downloaded input at $p = 32$ across input sizes. OpenMP shown as representative; all other middlewares show the same pattern.

Across all middlewares and both workloads, generated and downloaded inputs produce statistically indistinguishable runtimes. The timing curves overlap completely for the baseline, Pthreads, and OpenMP configurations at all input sizes. The MPI results show minor deviations at small n due to the known outlier runs at 32 processes, but these are measurement artifacts rather than input-dependent effects. This result confirms that FFT and FIR runtime is determined entirely by input length and not by signal content, which is expected given that both algorithms perform the same floating-point operations regardless of input values.

6 Conclusion

This project evaluated FFT and FIR filtering across Pthreads, OpenMP, and MPI on the Explorer Cluster, spanning ten input sizes from 2^{20} to 2^{29} samples and both synthetic and real-world audio inputs. The results demonstrate that algorithmic structure is the dominant factor governing parallel scaling behavior, outweighing middleware choice for workloads with inherent serial bottlenecks.

FIR filtering scales near-ideally across all middlewares. OpenMP and MPI both achieve

efficiency above 0.99 at 32 processes for large inputs, and weak scaling analysis confirms flat wall time as problem size grows proportionally with process count. The only exception is Pthreads, which degrades to approximately 0.41 efficiency at 32 threads due to thread management overhead that becomes significant relative to the per-thread computation at high thread counts.

FFT scaling is fundamentally limited by the serial `bit_reverse` permutation step, which VTune profiling shows accounts for 30 to 60 percent of CPU time regardless of thread count. Shared-memory middlewares are unable to overcome this bottleneck: Pthreads and OpenMP plateau at approximately 1.9x and 1.5x speedup respectively at 32 threads for $n = 2^{27}$, far below ideal. MPI mitigates this limitation through a segmented decomposition in which each process runs `bit_reverse` locally on N/p samples, achieving 19.4x speedup at 32 processes for the same input size.

The comparison between Pthreads and OpenMP reveals that programming model overhead is measurable but secondary. For FFT, both middlewares produce similar speedups despite the additional control that Pthreads provides. For FIR, OpenMP consistently outperforms Pthreads at high thread counts, suggesting that the OpenMP runtime's barrier implementation is more efficient than explicit pthread joins at 32 threads. This finding suggests that for inherent parallel workloads with no algorithmic bottleneck, the choice of middleware matters more than for bottleneck-bound workloads where the serial fraction dominates.

Future work could address the FFT scalability limitation by parallelizing the `bit_reverse` step itself, which would require bit manipulation to be distributed across threads, or by adopting a different FFT formulation that avoids the permutation step entirely. A CUDA GPU implementation would also be a natural extension, as the butterfly computation maps well onto GPU thread hierarchies and the large memory bandwidth of modern GPUs could mitigate the cache pressure observed at large input sizes.

Code Availability

The source code, benchmark scripts, result files, and analysis scripts for this project are available at:

<https://github.com/yalongwastaken/eece5640-finalproject>

Acknowledgments

The authors used Claude (Anthropic) as an AI assistant during the development of this project. AI assistance was used for debugging parallel implementations, reviewing analysis scripts, and editing portions of this report. All experimental results, analysis, and conclusions are the authors' own.

References

1. J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
2. OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, 2021.
3. Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, Version 4.0, 2021.
4. IEEE/The Open Group, *POSIX.1-2017: The Open Group Base Specifications Issue 7*, 2017.
5. V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An ASR corpus based on public domain audio books," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5206–5210, 2015.